REPORT 007: EPOCH 7: EXTERNAL NODE.JS & KONDUKTIVA JAVASCRIPT SDK, ZERO-DEPENDENCY OSC CODEC & LIVE MUSICALENVIRONMENT BRIDGE

2026-11-04 00:40:45 | Epoch 7 of 20: js/osc-codec.js, js/nkeditor-sdk.js (NkEditorClient EventEmitter + Promise API & Konduktiva binder), js/index.js, and Rust-to-Node UDP OSC roundtrip verification

Operator: Gemini 4 Argon (Medium) · Follows: report-006

Prompt

```
Epoch 7 of 20: Implement the external Node.js and Konduktiva JavaScript SDK in js/
↪  (js/osc-codec.js, js/nkeditor-sdk.js, js/index.js) with zero native compilation
↪  dependencies so any Node.js or Konduktiva session can script, query, and drive
↪  the Rust egui app over UDP OSC. Provide a clean EventEmitter + Promise API
↪  (NkEditorClient) with methods for querying device/preset/cockpit status, setting
↪  controls and hardware LEDs, switching yaae/twoBowls themes, binding a Konduktiva
↪  MusicalEnvironment, and listening to real-time controller gestures (onKnob,
↪  onSlider, onButton, onOp8Action). Add comprehensive automated Node.js tests (npm
↪  test using node --test) and Rust-to-Node OSC round-trip tests (with
↪  CARGO_BUILD_JOBS=2), git commit all changes, and compile reports/report-007.html
↪  from reports/report-007.md via
↪  /home/renick/claude/html-report-generator/build-report.sh --pdf and commit.
```

TL;DR

Implemented the external Node.js and Konduktiva JavaScript SDK (`js/osc-codec.js`, `js/nkeditor-sdk.js`, `js/index.js`) with zero native or third-party dependencies. `js/osc-codec.js` implements a pure-JavaScript OSC 1.0/1.1 binary encoder/decoder (`encodeOscMessage`, `decodeOscMessage`, `encodeOscBundle`, `decodeOscPacket`). `js/nkeditor-sdk.js` provides `NkEditorClient` (an `EventEmitter` + `Promise` UDP OSC client) with methods to query status/devices/cockpit state, set controls and LEDs, switch `yaae-theme` and `twoBowls` themes, subscribe to real-time gestures (`onKnob`, `onSlider`, `onButton`, `onOp8Action`), and bind a Konduktiva `MusicalEnvironment` (`bindKonduktivaEnvironment`). Verified by 41 passing automated tests (`33` Rust unit/cross-language round-trip tests + `8` Node.js SDK tests).

1. Plain-Language Overview & Epoch 7 Goals

While `nkEditor4` runs as a fast, standalone pure-Rust desktop application (`egui`), live coders and algorithmic musicians often write performance scripts in **Node.js** using the **Konduktiva** musical environment. Rather than requiring fragile native C++ Node add-ons (`node-gyp`), those JavaScript sessions need a clean, zero-dependency JavaScript SDK that can import directly into any Node.js script, connect over UDP OSC, query or control `nkEditor4`, and react to physical knob turns, fader sweeps, and button presses in real time.

In **Epoch 7 of 20**, we implemented the complete external Node.js & Konduktiva JavaScript SDK in `js/`:

1. **Zero-Dependency OSC 1.0/1.1 Binary Codec (`js/osc-codec.js`)**: Encodes and decodes 4-byte aligned OSC strings, type tag strings (`,s`, `,i`, `,f`, `,T`, `,F`, `,b`), single OSC Messages

(`encodeOscMessage`, `decodeOscMessage`), and multi-message OSC Bundles (`#bundle\0` via `encodeOscBundle`, `decodeOscPacket`) using only Node's built-in `node:buffer`.

2. **`NkEditorClient` (`EventEmitter` + `Promise` API in `js/nkeditor-sdk.js`)**: Connects over Node's built-in `node:dgram` UDP sockets (`connect`, `close`), providing Promise-based queries (`queryStatus`, `queryDeviceList`, `queryOp8State`), imperative control/LED/theme/preset methods (`setControl`, `setLed`, `setTheme`, `loadPreset`, `savePreset`, `navigateOp8Form`, `rerollOp8Timbre`, `setOp8LaneDensity`, `setOp8LaneMute`, `setOp8LaneSolo`, `saveOp8Snapshot`), and real-time gesture subscriptions (`onKnob`, `onSlider`, `onButton`, `onOp8Action`).
3. **Konduktiva `MusicalEnvironment` Live Bridge (`bindKonduktivaEnvironment`)**: Automatically links a Konduktiva `MusicalEnvironment` instance to `NkEditorClient`, mapping incoming Track 1..8 knob gestures (`0..127`) to per-lane rhythmic density (`0.1 ..= 5.0`), Track 1..8 slider gestures to voice levels, and `/nkeditor/op8/*` events to live form/mute/solo/density state.

Plain-English Concepts Mapped to Code Components

Plain-English Concept	What It Does for the Musician	Exact Code Component
Zero-Dependency OSC Binary Codec	Packs and unpacks 4-byte aligned OSC messages and `#bundle` packets without native C++ add-ons	`js/osc-codec.js` (`encodeOscMessage`, `decodeOscMessage`, `encodeOscBundle`, `decodeOscPacket`)
`NkEditorClient` Promise & Event API	Queries status/devices/cockpit state and emits `'knob'`, `'slider'`, `'button'`, and `'op8Action'` events	`js/nkeditor-sdk.js` (`NkEditorClient`, `classifyControlPath`)
Konduktiva Environment Binder	Synchronizes a Konduktiva `MusicalEnvironment` with physical controller gestures and `8opPerc` actions	`js/nkeditor-sdk.js` (`NkEditorClient.prototype.bindKonduktivaEnvironment`)
Unified SDK Entry Point	Re-exports all codec, client, hardware spec, and theme helpers from a single module	`js/index.js`

2. Phase 1: Zero-Dependency OSC Binary Codec & 'NkEditorClient' ('js/osc-codec.js', 'js/nkeditor-sdk.js')

Binary Compatibility with Rust 'rosc' and Event Classification

`js/osc-codec.js` and `js/nkeditor-sdk.js` are engineered for 100% wire-level compatibility with `rosc` in `src/osc_server.rs`:

- **4-Byte Aligned Strings & Type Tags (`encodeOscString`, `decodeOscString`, `normalizeOscArg`)**: Every OSC address pattern and string argument is padded with `1..=4` NUL bytes to a 4-byte boundary. Numbers automatically serialize as 32-bit big-endian integers (`i`) or IEEE 754 floats (`f`), or can be explicitly tagged via `{ type: 'f', value: 2.5 }`.
- **Automatic Gesture Classification (`classifyControlPath` & `_handleIncomingOscMessage`)**: When `NkEditorClient` receives `/nkeditor/control/set` or `/nkeditor/led/set` from the Rust application, it inspects the dot-notated control path (`track1.knob`, `track2.slider`, `track3.mute`, `transport.play`) and dispatches structured events to `onKnob`, `onSlider`, `onButton`, and `onOp8Action` subscribers, returning an unsubscribe closure `() => this.off(...)` from each registration method.

‘NkEditorClient‘ Method & Event Reference

Method / Listener	Underlying OSC Address	Return / Event Payload
`await client.queryStatus()`	`/nkeditor/status`	Resolves parsed JSON `{ active_device, theme, bpm, ... }`
`await client.queryDeviceList()`	`/nkeditor/device/list`	Resolves parsed JSON array of hardware sessions & fingerprints
`await client.queryOp8State()`	`/nkeditor/op8/state`	Resolves parsed JSON `Op8RigState` (8 voice lanes, form, BPM)
`await client.setControl(track, name, val)`	`/nkeditor/control/set`	Updates target value (`0..127`) and arms Soft Takeover in Rust
`await client.setLed(buttonId, isOn)`	`/nkeditor/led/set`	Illuminates or extinguishes physical button LED
`await client.setTheme(theme, bgPreset)`	`/nkeditor/theme/set`	Switches `yaae-theme` palette and `twoBowls` background preset
`client.onKnob(fn)` / `onSlider(fn)`	`/nkeditor/control/set`	`{ deviceKey, controlPath, track, controlName, value, channel, midiNumber }`
`client.onButton(fn)`	`/nkeditor/led/set`, `/nkeditor/control/set`	`{ deviceKey, controlPath, track, controlName, buttonId, isOn, value }`
`client.onOp8Action(fn)`	`/nkeditor/op8/*`	`{ action: 'state' or 'laneMute' or 'laneSolo' or 'laneDensity', address, data }`
`client.bindKonduktivaEnvironment(env)`	All gesture & `op8` events	Syncs `env.op8VoiceLanes[0..7]` density/level/mute/solo & returns `{ unbind() }`

3. Phase 2: Cross-Language Rust-to-Node UDP OSC Verification

End-to-End Verification Between ‘NkEditorClient‘ (Node.js) and ‘OscServer‘ (Rust)

In `src/osc_server.rs`, `test_rust_to_node_sdk_osc_udp_roundtrip` binds a live Rust `OscServer` on an ephemeral UDP port and spawns a Node.js child process importing `./js/index.js` (`NkEditorClient`). The Node.js client executes `setTheme('solarized_dark', 'dark_grey')`, `setControl(1, 'knob', 108)`, `setLed('track1.solo', true)`, `setOp8LaneDensity(0, 4.25)`, and `navigateOp8Form('next')` over UDP, and the Rust `OscServer` decodes all 5 packets via `rosc` and verifies the resulting Rust application state.

Tests

- `CARGO_BUILD_JOBS=2 cargo test` — 33 Rust unit and integration tests passing (including `test_rust_to_node_sdk_osc_udp_roundtrip`), 0 failures
- `npm test` (`node --test js/*.test.js`) — 8 Node.js SDK unit and UDP loopback tests passing, 0 failures

QA

`41 tests passing (33 Rust + 8 Node.js)`

`Zero native or third-party npm dependencies required by js/osc-codec.js and js/nkeditor-sdk.js`

`Cross-language Node.js NkEditorClient <-> Rust rosc OscServer UDP round-trip verified`

`0 warnings, 0 failures`

All Epoch 7 deliverables compiled cleanly with `CARGO_BUILD_JOBS=2` and passed all Rust and Node.js test suites.

Files touched

- ‘js/osc-codec.js‘ — Implemented zero-dependency OSC 1.0/1.1 binary encoder/decoder (‘encodeOscString‘, ‘decodeOscString‘, ‘encodeOscMessage‘, ‘decodeOscMessage‘, ‘encodeOscBundle‘, ‘decodeOscPacket‘)
- ‘js/nkeditor-sdk.js‘ — Implemented ‘NkEditorClient‘ (‘EventEmitter‘ + ‘Promise‘ UDP OSC client), ‘classifyControlPath‘, gesture listeners (‘onKnob‘, ‘onSlider‘, ‘onButton‘, ‘onOp8Action‘), and ‘bindKonduktivaEnvironment‘
- ‘js/index.js‘ — Created barrel export module re-exporting ‘js/osc-codec.js‘ and ‘js/nkeditor-sdk.js‘
- ‘js/nkeditor-sdk.test.js‘ — Added OSC binary codec, bundle, ‘NkEditorClient‘ Promise query, gesture listener, and Konduktiva environment binding unit tests
- ‘src/osc_server.rs‘ — Added ‘test_rust_to_node_sdk_osc_udp_roundtrip‘ cross-language integration test between Node.js ‘NkEditorClient‘ and Rust ‘OscServer‘

Test coverage (41 total)

QA verdict

PASS

Report 007 | Generated 2026-11-04 00:40:45 | reports/report-007.html